FAST · arXiv 2501.09747 · Physical Intelligence

Visionary

11+3 · seat 5

Hybrid for 2026: FM(+RTC) deploy
FAST pretrain · no Discord/Drive

Oct 19 paper club · VLA Architectures

MYTH BUST: FAST ≠ FM · $\pi_{0.5}$ = FAST pretrain → FM deploy

π_0 ≈ FM · π_0-FAST ≈ discrete AR · $\pi_{0.5}$ ≈ hybrid successor

CRITICAL MYTH BUST · say this first

FAST does NOT replace flow matching at deploy

Label	Meaning
FAST	Discrete DCT→BPE action tokenizer for AR VLAs
FM	Continuous flow-matching (π_0 / $\pi_{0.5}$ deploy)
π_0-FAST	Discrete AR variant using this tokenizer
$\pi_{0.5}$	FAST for pretrain · FM for post-train/deploy

Naming: π_0 ≈ FM · π_0-FAST ≈ discrete AR · $\pi_{0.5}$ ≈ hybrid successor

Hybrid for 2026 BEHAVIOR

FM(+RTC) deploy · FAST pretrain/aux · no Discord/Drive

1

Keep FM(+RTC)

Smooth multi-second household skills + latency at serve

2

Keep FAST

Cheap AR co-train, language, tokenizer compression research

3

AR decode speed

Speculative decode / quant / draft if discrete VLAs compete dynamically

4

FAST+ default

New morphologies / sims entering challenge without training VQ

5

Aux loss pattern

B1K'25: FAST aux w=0.05 on Pi0.5 FM primary — hybrid wins

Skip Discord/Drive — cite paper + HF + PI blog + $\pi_0/\pi_{0.5}$ packs

When FAST helps vs when continuous FM wins

FAST helps

- High-Hz next-token prediction
- Train compute / compression
- Denser chunks without token blowup
- Language-following AR VLAs
- Cheap large-scale discrete pretrain
- $\pi_{0.5}$ discrete pretrain phase

Continuous FM wins

- Inference latency / smooth dynamics
- Closed-loop control (fewer ms/chunk)
- Real-time household deploy
- ~100 ms vs ~750 ms on 4090
- π_0 / $\pi_{0.5}$ post-train & serve
- Contact-rich multimodal density

$\pi_{0.5}$ recipe: FAST for efficient pretrain NTP → serve with continuous FM

Naming hygiene tree · Archaeologist / club spine

Never collapse these three labels

π_0

≈ continuous FM
PaliGemma + action expert
~100 ms / chunk deploy

π_0-FAST

≈ discrete AR sibling
Same backbone + FAST tokens
~750 ms / chunk · 5× train

$\pi_{0.5}$

≈ hybrid successor
FAST pretrain → FM post
FM RETAINED at deploy

Forward lineage (CLEAR)

FAST → **π_0-FAST** → **$\pi_{0.5}$ (FAST→FM)** → **KI** → **RTC** → **BEHAVIOR**

Inference latency · RTX 4090 class

~100 ms FM vs ~750 ms AR-FAST per 1 s chunk (~7.5×)

Static manip OK · dynamic closed-loop / eval throughput hurt

Future: speculative decode · quant · kernels · Empiricist must log ms/chunk

Club one-liner

FAST = discrete DCT→BPE tokenizer for AR VLAs.

π_0-FAST matches FM π_0 on dexterity at ~5× less train compute,

but infer ~750 ms vs FM ~100 ms.

$\pi_{0.5}$ keeps FAST for pretrain and FM for deploy.

Do NOT delete FM because FAST exists.

Stakeholder decision · train vs deploy

USE FAST+

AR training / cheap pretrain
Drop-in HF tokenizer
π_0-FAST when AR research
.fit() for weird morphologies

KEEP FM

Realtime closed-loop deploy
π_0 / $\pi_{0.5}$ OpenPI path
~100 ms class latency
Do NOT delete because FAST exists

Risk call-out

AR inference latency ~7.5× vs FM (paper 4090: ~750 ms vs ~100 ms)

γ reconstruction loss · FAST+ mix leakage caveat on eval robots · private 10k h unreproducible

Adopt FAST+/π_0-FAST as discrete branch; keep FM as deploy branch unless latency is solved

FAST: Efficient Action Tokenization for Vision-Language-Action Models

Karl Pertsch[*,1,2,3], Kyle Stachowicz[*,2],
Brian Ichter[1], Danny Driess[1], Suraj Nair[1], Quan Vuong[1], Oier Mees[2], Chelsea Finn[1,3], Sergey Levine[1,2]

[1]Physical Intelligence, [2]UC Berkeley, [3]Stanford

https://pi.website/research/fast

arXiv:2501.09747v1 [cs.RO] 16 Jan 2025

***Abstract*—Autoregressive sequence models, such as Transformer-based vision-language action (VLA) policies, can be tremendously effective for capturing complex and generalizable robotic behaviors. However, such models require us to choose a tokenization of our continuous action signals, which determines how the discrete symbols predicted by the model map to continuous robot actions. We find that current approaches for robot action tokenization, based on simple per-dimension, per-timestep binning schemes, typically perform poorly when learning dexterous skills from high-frequency robot data. To address this challenge, we propose a new compression-based tokenization scheme for robot actions, based on the discrete cosine transform. Our tokenization approach, Frequency-space Action Sequence Tokenization (FAST), enables us to train autoregressive VLAs for highly dexterous and high-frequency tasks where standard discretization methods fail completely. Based on FAST, we release FAST+, a *universal* robot action tokenizer, trained on 1M real robot action trajectories. It can be used as a black-box tokenizer for a wide range of robot action sequences, with diverse action spaces and control frequencies. Finally, we show that, when combined with the π_0 VLA, our method can scale to training on 10k hours of robot data and match the performance of diffusion VLAs, while reducing training time by up to 5x.**

I. Introduction

Large, high-capacity Transformer models can be tremendously effective for capturing complex and generalizable robotic behaviors both from scratch [8, 69, 51, 6, 20, 62] and using models pre-trained for next-token prediction on Internet-scale image-text corpora [10, 39, 63, 7, 65]. However, these models require choosing a tokenization of the continuous action signal, which determines how the discrete symbols predicted by the model map to continuous robot actions [64, 34, 41, 12]. It is widely known that a good choice of tokenization can be critical to the performance of sequence models [55, 57]. Prior robotic policies of this sort typically use naïve tokenization strategies based on a per-dimension, per-timestep binning scheme [9, 10, 39]. We find that such methods perform poorly when learning dexterous skills with high-frequency control (see Figure 2, right). We observe that correlations between time steps are a major challenge for naïve tokenization strategies when predicting sequences of

*: Core contributors
Correspondence to: research@physicalintelligence.company

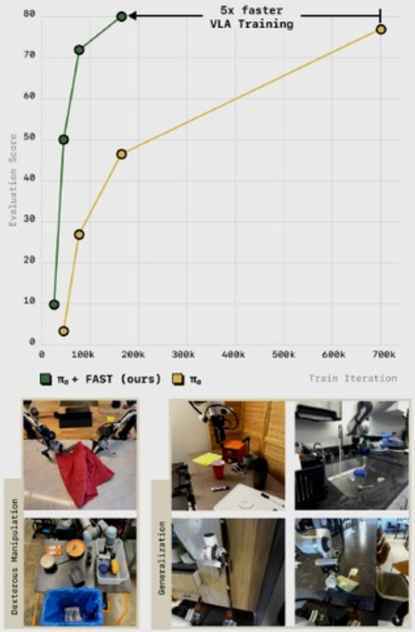

Fig. 1: We propose FAST, a simple yet effective approach for tokenization of robot action trajectories via time-series compression. FAST enables training of autoregressive VLAs that solve complex dexterous manipulation tasks and generalize broadly to new scenes. We use it to train π_0-FAST, a generalist robot policy that matches the performance of the state-of-the-art π_0 diffusion VLA on dexterous and long-horizon manipulation tasks, while training 5x faster (**top**).

Fig. 10: **Rollout of π_0-FAST on the laundry folding task.** FAST tokenization enables autoregressive VLAs to perform complex, long-horizon, and dexterous tasks that were impossible with previous tokenization schemes.

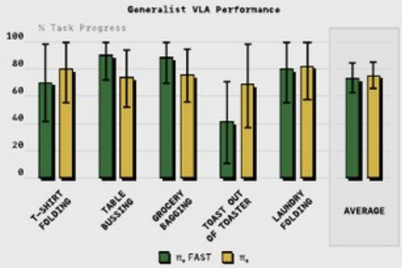

Fig. 11: **Comparison of π_0-FAST and diffusion π_0 [7] generalist policies.** π_0-FAST matches the performance of diffusion π_0 while requiring significantly less compute for training. Reported: mean and 95% CI.

F. Scaling Autoregressive VLAs to Large Robot Datasets

We have demonstrated FAST's effectiveness for training autoregressive VLAs on individual robot datasets, but does it scale to training dexterous *generalist* policies? To test this, we train the π_0-FAST model from the previous section on the cross-embodied robot data mixture used by π_0 [7], the largest dexterous robot manipulation dataset to date. It includes 903M timesteps from our own datasets. Additionally, 9.1% of the training mixture consists of the open-source datasets BRIDGE v2 [60], DROID [38], and OXE [52].

We compare zero-shot performance to the diffusion π_0 model on the tasks from Black et al. [7] in Figure 11. Overall, we find that the autoregressive π_0-FAST model matches the performance of the diffusion π_0 model, including on the most challenging *laundry folding* task, **while requiring significantly less compute for training**. We show a qualitative example of π_0-FAST performing the laundry folding task in Figure 10 and include additional videos on our website.

Importantly, we find that π_0-FAST converges significantly faster than the diffusion π_0 model: the model in the evaluations above required 5x fewer GPU hours for training than the π_0 model from Black et al. [7]. We show robot evaluation results for multiple checkpoints throughout the course of training in Figure 1 (averaging performance on two representative tasks: table bussing and t-shirt folding). The results show clearly that π_0-FAST achieves high performance significantly faster. For state-of-the-art VLA training runs, which can often use thousands of GPU hours, a 5x reduction in required compute is significant. We include a full comparison across all tasks for a compute-matched π_0 checkpoint in Appendix, Figure 15 and find that the same conclusions hold: π_0-FAST clearly outperforms compute matched π_0 due to its faster convergence.

To summarize, we have demonstrated that FAST tokenization allows us to train autoregressive VLAs on complex, dexterous robot tasks that prior tokenization schemes completely fail on. We have also shown that FAST, when combined with state-of-the-art VLAs like π_0, scales to training generalist, cross-embodied policies that rival the performance of the best diffusion VLAs while being significantly faster to train.

VII. Discussion and Future Work

In this paper, we introduced FAST, an efficient action tokenizer for high-frequency robotic control data. FAST uses the discrete cosine transform (DCT) followed by byte-pair encoding (BPE) to compress action chunks, leading to significantly better compression than existing action tokenizers across a range of robotics domains. Our real-world and simulated VLA experiments show that FAST leads to dramatically improved performance over the previously used naïve action discretization approaches, and outperforms more complex learned tokenization methods based on vector quantization. We also showed that we can train FAST+, a *universal* action tokenizer, that can serve as a strong default tokenizer for any robot action sequence. Using it, we trained π_0-FAST, a dexterous generalist policy that can match performance of

Empiricist smoke matrix · priority grid

P0 Y · P1 AR-FAST vs continuous FM · Defer full 10k h

P	Experiment	Smoke
P0	HF FAST+ encode/decode; compression vs naïve; didactic AR toy across Hz	Y
P1	VLA FT FAST+ vs bins; BPE on/off; γ sweep; AR-FAST vs continuous FM	Partial
P2	fit() vs universal; $\pi_{0.5}$ hybrid echo (short FAST→FM)	Partial
Defer	Full π_0-FAST 10k h; real laundry hardware	N

Env outline: /workspace/fast-quiet/env_outline/

Log: tokens/chunk · recon MSE · train steps · success · ms/chunk · GPU name · max_memory_allocated

FAST pipeline · DCT → BPE

Quantile norm → DCT → scale γ → round → low-freq-first flatten → BPE |V|=1024

① Norm

1–99% quantile
→ [-1,1]

② DCT

per-dim
over H

③ Quant

round(γ·C)
γ=10 default

④ Flatten

low-freq
across dims first

⑤ BPE

vocab 1024
~30 tok/arm

Why it works

- Naïve bins at high Hz → copy-last-token
- DCT packs shape into low-freq coeffs
- Sparse zeros → BPE squashes stream
- Invertible decode → continuous actions
- Overwrite least-used VLM vocab slots

Defaults / HF

Chunk ≈ 1 second

γ = 10 · |V| = 1024

Pad dims to 32 (FAST+)

```
AutoProcessor.from_pretrained(
  "physical-intelligence/fast",
  trust_remote_code=True)
```

Visionary · close

2026: FM(+RTC) at deploy · FAST for pretrain/aux · invest AR decode speed.

Skip Discord/Drive · B1K winners already use FAST aux on Pi0.5 FM

Q&A · Oct 19 · FAST / 2501.09747

Hybrid for 2026 BEHAVIOR

FM(+RTC) deploy · FAST pretrain/aux · no Discord/Drive

1

Keep FM(+RTC)

Smooth multi-second household skills + latency at serve

2

Keep FAST

Cheap AR co-train, language, tokenizer compression research

3

AR decode speed

Speculative decode / quant / draft if discrete VLAs compete dynamically

4

FAST+ default

New morphologies / sims entering challenge without training VQ

5

Aux loss pattern

B1K’25: FAST aux w=0.05 on Pi0.5 FM primary — hybrid wins

Skip Discord/Drive — cite paper + HF + PI blog + $\pi_0/\pi_{0.5}$ packs